On those platforms that support a double-word-compare-and-swap operation, this structure will be small enough for `std::atomic<counted_node_ptr>` to be lock-free. If it isn't on your platform, you might be better off using the `std::shared_ptr<>` version from listing 7.9, because `std::atomic<>` will use a mutex to guarantee atomicity when the type is too large for the platform's atomic instructions (thus rendering your "lock-free" algorithm lock-based after all). Alternatively, if you're willing to limit the size of the counter, and you know that your platform has spare bits in a pointer (for example, because the address space is only 48 bits but a pointer is 64 bits), you can store the count inside the spare bits of the pointer to fit it all back in a single machine word. Such tricks require platform-specific knowledge and are thus outside the scope of this book.

`push()` is relatively simple ❺. You construct a `counted_node_ptr` that refers to a freshly allocated `node` with associated data and set the `next` value of the `node` to the current value of `head`. You can then use `compare_exchange_weak()` to set the value of `head`, just as in the previous listings. The counts are set up so the `internal_count` is zero, and the `external_count` is one. Because this is a new node, there's currently only one external reference to the node (the `head` pointer itself).

As usual, the complexities come to light in the implementation of `pop()`, which is shown in the following listing.

Listing 7.11. Popping a node from a lock-free stack using split reference counts